

Topics to cover

1. Middleware
2. Request Lifecycle
3. How FastAPI Processes a Request

① The Request Lifecycle

When a client send a request:

1. Request enters server
2. Middleware run (before)
3. Routing decides which endpoint to call
4. Dependencies are resolved (depends)
5. Your endpoint function runs
6. Response is created
7. Middleware run (after)
8. Response is sent back

Analogy (Airport):

- ① Security check = Middleware (before)
- ① Gate selection = Router
- ① Boarding pass check = Dependencies
- ① Flight = Your endpoint logic
- ① Exit Customs = Middleware (after)



② What is Middleware?
Middleware = Code that runs for every request and every response.

Uses for:

- ① Logging
- ① Authentication
- ① Timing request
- ① Adding headers
- ① CORS
- ① Rate limiting

③ Create a Middleware
From FastAPI: `import FastAPI, Request`
`import time`

```
app = FastAPI()
```

```
@app.middleware("http")
```

```
async def log_requests(request: Request, call_next):
```

```
    start_time = time.time()
```

```
    response = await call_next(request)
```

```
    process_time = time.time() - start_time
```

```
    print(f"{request.method} {request.url.path} took {process_time:.4f}s")
```

```
    return response
```

What is Happening here?

1. A request comes in
example

GET /users

Before it reaches your route function,



the middleware runs.

2. Start Timer

`start-time = time.time()`

You store the current time.

3. Pass Request Forward

`response = await call-next(request)`

This line is very important.

It means:

"Continue processing the request".

FASTAPI now:

- Runs the route function

- Get the response

- Return it back here

Without this line, your request would stop.

④ Calculate Processing time

`process-time = time.time() - start-time`

This calculate how long the request took.

⑤ Print Log

`Print(f'{request.method} {request.url.path} took {process-time: .4f}s')`

Example output:

GET /users took 0.0234s



⑥ Return Response

- return response
- ⑥ response is sent back to client.

Middleware runs:

Before request → Route runs → After route → Response sent

Think of it like Sandwich:

[Middleware Before]

↓

Route

↓

[Middleware After]

CORS

Cross origin Resource sharing

It control:

- ⑥ Which Frontend domain can call your API

In FastAPI:

```
from fastapi.middleware.cors
import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins = ["*"],
    allow_methods = ["*"],
    allow_headers = ["*"],
)
```